

data are often in a format not suited for the learning...

data **preprocessing** may help in improving the prediction model

- image transformation (e.g. pixel matrix for handwritten digit)
- categorical data transformations → **MACHINE LEARNING EXPECT NUMERICAL INPUTS...**
 - ▶ bad method: transform categories into integer numbers
L'Aquila → 1, Rome → 2, Milan → 3
inherits a meaningless order (L'Aquila < Rome < Milan)
 - ▶ good method: **one-hot encoding** transforms categorical data in array with a number of binary elements equal to the number of categories (-1)
L'Aquila → [1 0], Rome → [0 1], Milan → [0 0]
features grow too much with many categories
- text data transformations
 - ▶ **words count**: count the number of occurrences of each word in a text
sample = ['problem of evil', 'evil queen', 'horizon problem']

LESS EFFICIENT

L'AQUILA	1	0
ROME	0	1
MILAN	0	0

→

	evil	horizon	of	problem	queen
1	1	0	1	1	0
1	1	0	0	0	1
0	0	1	0	1	0

- derivations (e.g. polynomial transformation)
- NA values imputations → **NOT AVAILABLE** → **REPLACE THEM**
- standardization/normalization (**target normalization** very important in **regression!**)

sklearn.preprocessing, sklearn.feature_extraction



UNIVERSITA
DELLA
EGE

feature selection may be useful to limit the complexity by reducing the actually used features

all possible subsets of features are 2^n , too expensive training 2^n models...

...heuristics!

- **forward selection**

↗ NO FEATURES

- 1 start with the null-model, train all n single feature models and add to the null-model the feature corresponding to the best performing single feature model
- 2 train all the $n - 1$ two features models and add to the previous single features model the feature corresponding to the best performing two features model
- 3 iterate until some stopping condition is reached

MAY BE RELATED TO

- **backward selection**

- 1 start with whole n feature model and train it
- 2 iteratively remove the feature which is "statistically less significant" until some stopping condition is reached

- MAXIMUM NUMBER OF FEATURES
- MODEL PERFORMANCE
- NO SIGNIFICANT IMPROVEMENT

- **mixed selection**: combine forward and backward selection

`sklearn.feature_selection`



complexity can be also controlled by adding a **regularization** term to the loss function

example: multivariate regression

$\lambda \geq 0$ REGULARIZATION COEFFICIENT
TRADE-OFF ERROR/COMPLEXITY

$$\min_{\beta} \underbrace{\text{RSS}}_{\text{ERROR}} + \lambda \cdot \underbrace{\text{regularization}}_{\text{COMPLEXITY}} = \sum_{i=1,m} \underbrace{(y^i - \hat{y}^i)^2}_{\text{RSS}} + \lambda \|\beta\|_p^p$$

typical choices

- $p = 1$: **lasso** regression $\rightarrow \min \sum_{i=1,m} (y^i - \hat{y}^i)^2 + \lambda \sum_{j=0,n} |\beta_j|$
- $p = 2$: **ridge** regression $\rightarrow \min \sum_{i=1,m} (y^i - \hat{y}^i)^2 + \lambda \sum_{j=0,n} \beta_j^2$

REDUCING THE NORM OF THE PARAMETERS OF A MODEL IMPLIES REDUCE THE COMPLEXITY

LASSO

- SPARSITY (MANY 0 PARAMETERS)
- CONVEX
- NOT-DIFFERENTIABLE

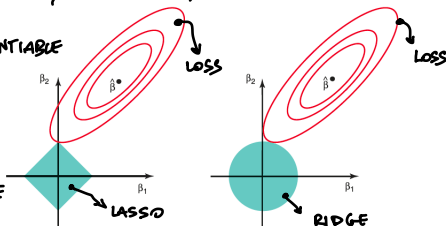
RIDGE

- NO SPARSITY
- CONVEX
- DIFFERENTIABLE
- MORE SMOOTH MODELS

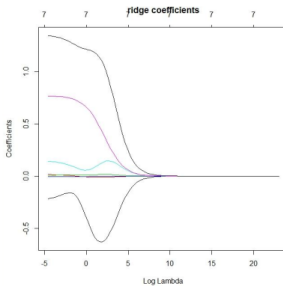
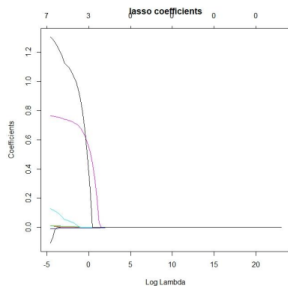
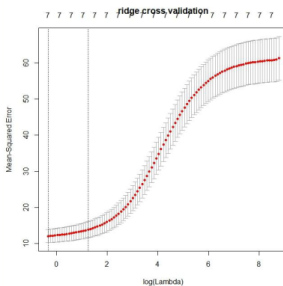
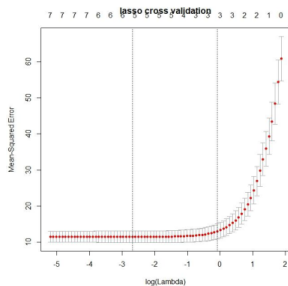
MODELS → BETTER GENERALIZATION

lasso ⇒ more **sparse** models

ridge ⇒ more **smooth** models



example: grid-search cross-validation results for lasso regression and ridge regression



check the dataset quality:

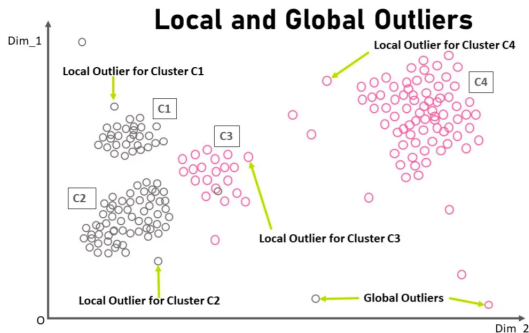
- are there missing values? How to remove them?
 - ▶ remove rows with missing values
 - ▶ replace missing values with representative values (mean, mode, median,...)
 - ▶ replace missing values with "close" values
- are data informative?
 - ▶ collinearity and multicollinearity
- are there anomalous values?
 - ▶ outlier-novelty detection

outliers detection → STRONGLY AFFECTS PERFORMANCE

let's focus on anomalies... → NOISY

anomalies are "polluted" samples in the training set (**outliers**) or in unseen observations (**novelties**)

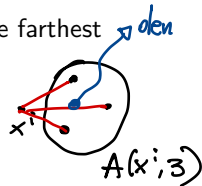
- **global outliers**: samples significantly different from the rest of the dataset
- **local outliers**: samples significantly different from their neighbours



Local Outlier Factor (LOF) $\longrightarrow$ OUTLIER SCORE

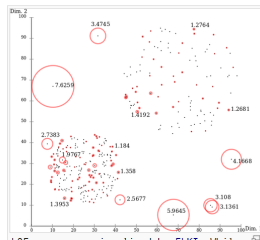
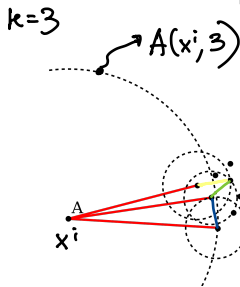
- consider set $A(x^i; K)$ of observations of the **K nearest neighbors** from a sample x^i
- compute the **density** of x^i as its ~~maximum~~ distance from the farthest observation in $A(x^i; K)$:

$$\text{den}(x^i; K) = \max_{x^j \in A(x^i; K)} d(x^i, x^j)$$



- if ratio $\text{LOF}(x^i; K) = \frac{\text{den}(x^i; K)}{\frac{1}{|A(x^i; K)|} \sum_{x^j \in A(x^i; K)} \text{den}(x^j; K)} \geq \eta \gg 1 \implies x^i$ is an outlier

$\downarrow$
THRESHOLD

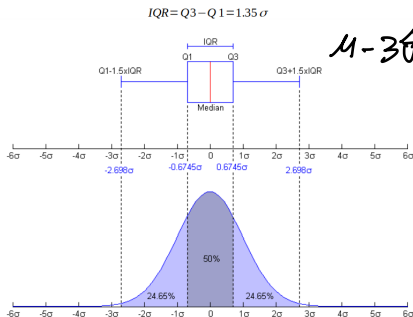


LOF considerations

- it is able to determine local outliers and not only global
 - performance affected by K
 - performance affected by η
- $\searrow$ HYPERPARAMETERS

by assuming normally distributed data **σ -clipping** can be used to detect outliers

- for normal distributions $IQR = 1.35\sigma$
 - σ -clipping idea: exploit **observed** Q1 and Q3 to estimate σ and construct confidence interval for not outliers (**inliers**)
- $\searrow \hat{\sigma}$



$$\mu - 3\hat{\sigma} \leq x \leq \mu + 3\hat{\sigma}$$

exercise 4.2